

ASSIGNMENT 14

Name: Rajdeep Das

Roll No: CSB25203

Subject: Advanced Programming

```
import sys
```

```
import gc
```

```
class Node:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        self.link = None
```

```
    def __del__(self):
```

```
        print(f"{self.name} is being destroyed")
```

```
# Enable automatic garbage collection
```

```
gc.enable()
```

```
print("=== Creating Nodes ===")
```

```
A = Node("Node A")
```

```
B = Node("Node B")
```

```
# Create cyclic references
```

```
A.link = B
```

```
B.link = A
```

```
print("Cycle created:")
```

```
print("A ->", A.link.name)
```

```
print("B ->", B.link.name)
```

```
print("\n=== Reference Counts ===")
```

```
# sys.getrefcount() adds one temporary reference itself
```

```
print("Reference count for A:", sys.getrefcount(A))
```

```
print("Reference count for B:", sys.getrefcount(B))
```

```
print("\n=== Objects Currently Tracked by GC ===")
```

```
for obj in gc.get_objects():
```

```
    if isinstance(obj, Node):
```

```
        print("Tracked object:", obj.name)
```

```
print("\n=== Deleting External References ===")
```

```
# Remove the only external references
```

```
del A
```

```
del B
```

```
print("Variables A and B deleted")
```

```
print("\n=== Investigating Memory Before Collection ===")
```

```
# Objects still exist because they reference each other
```

```
found_nodes = 0
```

```
for obj in gc.get_objects():
    if isinstance(obj, Node):
        found_nodes += 1
        print("Still in memory:", obj.name)

print("Total Node objects still tracked:", found_nodes)

print("\n=== Forcing Garbage Collection ===")

unreachable = gc.collect()

print("Unreachable objects collected:", unreachable)

print("\n=== Memory After Collection ===")

found_nodes = 0

for obj in gc.get_objects():
    if isinstance(obj, Node):
        found_nodes += 1
        print("Still in memory:", obj.name)

print("Remaining Node objects:", found_nodes)
```